



UNIVERSIDADE FEDERAL DA PARAÍBA
CAMPUS IV - RIO TINTO
SISTEMAS DE INFORMAÇÃO

Equipe: Rhauany Aragão, Jamilly Santos, Maria Rafaela, Mariano Santos.

Disciplina: Análise e Projeto de Sistemas

Professor: Dorgival Pereira da Silva Netto

Tema: Sistema de gestão de granja

Agro Plus

Padrões de Projeto – AgroPlus

1. Introdução

O sistema AgroPlus foi desenvolvido utilizando a arquitetura MVC (Model-View-Controller), visando separar responsabilidades entre interface, regras de negócio e persistência de dados. Para aumentar a modularidade, reutilização e facilidade de manutenção do software, foram adotados padrões de projeto baseados nos princípios GRASP e nos padrões GoF.

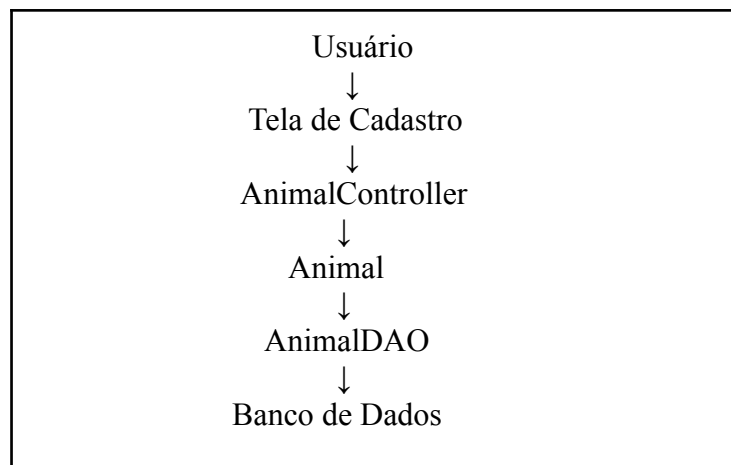
2. Padrões GRASP

Os padrões GRASP foram utilizados para distribuir as responsabilidades entre as classes do sistema AgroPlus de forma organizada, reduzindo o acoplamento e facilitando a manutenção do software.

2.1 Controller

O padrão Controller é responsável por receber as requisições realizadas pelo usuário por meio da interface, processar essas solicitações e encaminhá-las para as classes do modelo responsáveis pelas regras de negócio.

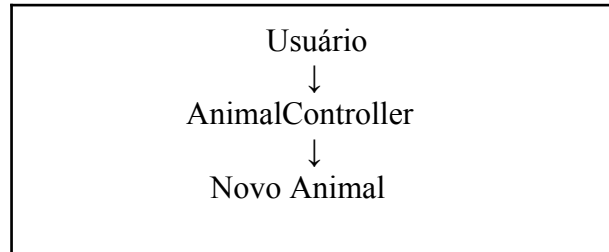
No sistema AgroPlus, as classes `AnimalController`, `EstoqueController` e `ProducaoController` serão responsáveis por intermediar a comunicação entre a interface (View) e as classes do Model. Quando o usuário desejar cadastrar um novo animal, a requisição será enviada ao `AnimalController`, que validará as informações recebidas e solicitará ao modelo a criação do objeto `Anima`, por exemplo.



A utilização do Controller evita que a interface tenha acesso direto às regras de negócio e ao banco de dados, centralizando o processamento das ações do sistema e tornando o código mais organizado e garantindo, dessa forma, a separação entre interface e lógica de negócio, melhor organização do sistema e facilidade de manutenção e evolução.

2.2 Creator

O padrão Creator determina qual classe deve ser responsável pela criação de novos objetos, no caso do nosso sistema, as classes Controller serão responsáveis pela criação das entidades do sistema, pois recebem todas as informações fornecidas pelo usuário durante o cadastro.

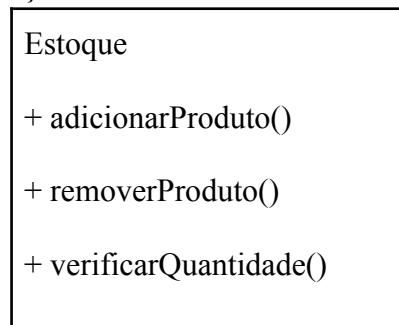


Como o Controller possui todas as informações necessárias para construir o objeto, ele é o responsável mais adequado para realizar essa criação. Os benefícios são: responsabilidades bem definidas, código mais organizado e facilidade para futuras alterações.

2.3 Information Expert

O padrão Information Expert estabelece que a responsabilidade por uma operação deve ser atribuída à classe que possui as informações necessárias para executá-la.

No nosso sistema, a classe Estoque será responsável pelas operações relacionadas ao controle dos produtos armazenados, como atualizar quantidades disponíveis, verificar disponibilidade e registrar entradas e saídas. Da mesma forma, a classe Produção será responsável pelo cálculo da produção diária, enquanto a classe Animal armazenará e gerenciará as informações referentes aos animais cadastrados.

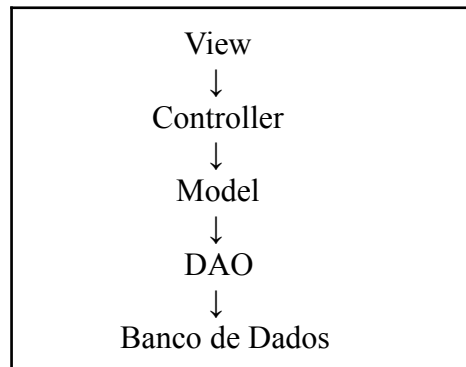


Cada classe executa apenas as operações relacionadas às informações que ela própria possui, evitando que outras classes assumam responsabilidades indevidas, o que garante melhor encapsulamento, redução da duplicação de código e maior facilidade de manutenção.

2.4 Low Coupling

O padrão Low Coupling busca reduzir a dependência entre as classes do sistema, facilitando alterações futuras e aumentando a reutilização do código. No AgroPlus, Controllers não acessam diretamente o banco de dados. Toda comunicação com a

persistência é realizada por meio das classes DAO, enquanto as regras de negócio permanecem nas classes do Model.



Ao reduzir o nível de dependência entre as classes, torna-se possível modificar uma parte do sistema sem causar impactos significativos nas demais e assegura um código modular, mais facilidade para manutenção, maior reutilização de componentes e melhor escalabilidade do sistema.

A utilização dos padrões GRASP permitiu distribuir corretamente as responsabilidades entre as classes e com isso o sistema apresenta maior organização, menos acoplamento entre os componentes e melhor capacidade de manutenção e evolução, além de complementar a arquitetura MVC adotada no projeto.